REVISION CHEAT SHEET:

SQL SERVER FUNCTIONS AND EXPRESSIONS

Interview-ready notes • examples • SQL • diagrams

SQL Server provides many built-in functions that allow us to transform, calculate, compare, format, and analyze values.

Before learning the individual functions, understand two basic terms:

EXAMPLE / FLOW

```
Expression
Function
```

What Is an Expression?

An **expression** is a combination of values, columns, operators, and functions that produces a result.

Simple example:

SQL

```
SELECT 10 + 20;
```

Result:

EXAMPLE / FLOW

```
30
```

Here:

EXAMPLE / FLOW

```
10 + 20
```

is an expression.

Another example:

SQL

```
SELECT Salary * 12
FROM Employees;
```

Here:

EXAMPLE / FLOW

```
Salary * 12
```

is an expression that calculates annual salary.

Expressions can contain:

EXAMPLE / FLOW

```
Constants
Columns
Operators
Functions
CASE expressions
Other expressions
```

Example:

SQL

```
SELECT
    FirstName + ' ' + LastName AS FullName
FROM Employees;
```

What Is a Function?

A **function** performs an operation and returns a value.

Example:

SQL

```
SELECT UPPER('john');
```

Result:

EXAMPLE / FLOW

```
JOHN
```

Here:

EXAMPLE / FLOW

```
UPPER()
```

is a function.

Basic idea:

EXAMPLE / FLOW

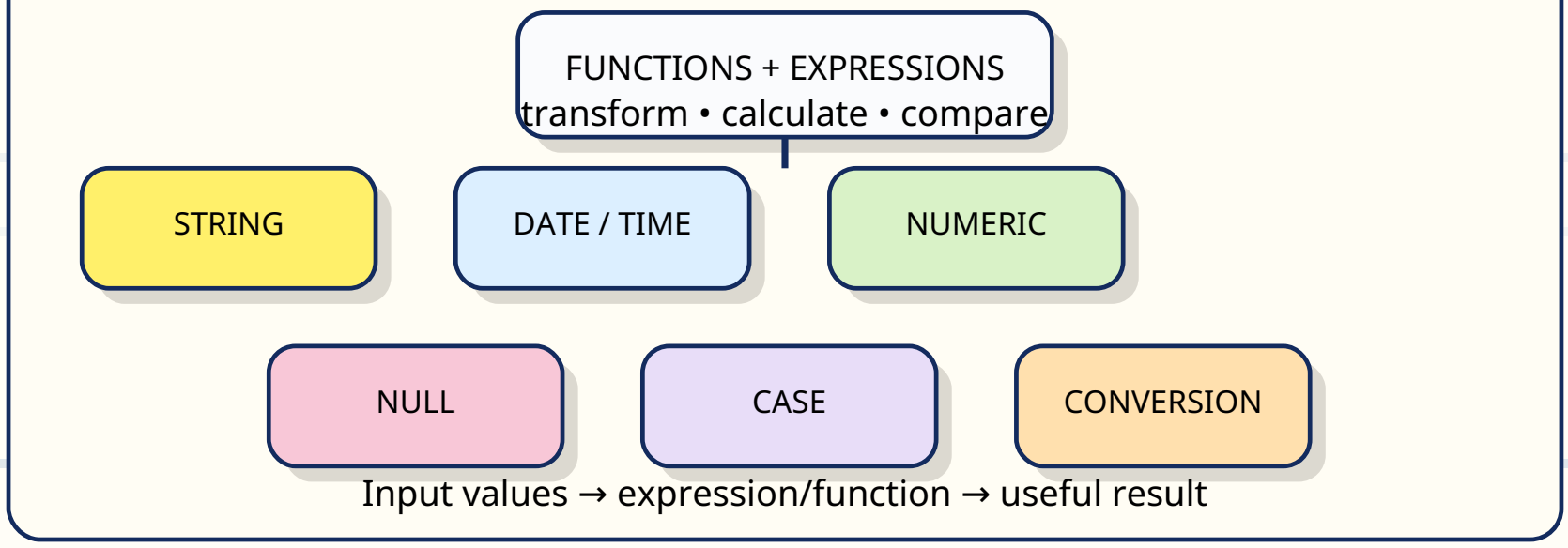
```
Input
  ↓
Function
  ↓
Output
```

Example:

EXAMPLE / FLOW

```
john
 ↓
UPPER()
 ↓
JOHN
```

Main Function Groups in This Note



EXAMPLE / FLOW

```
String Functions

Date and Time Functions

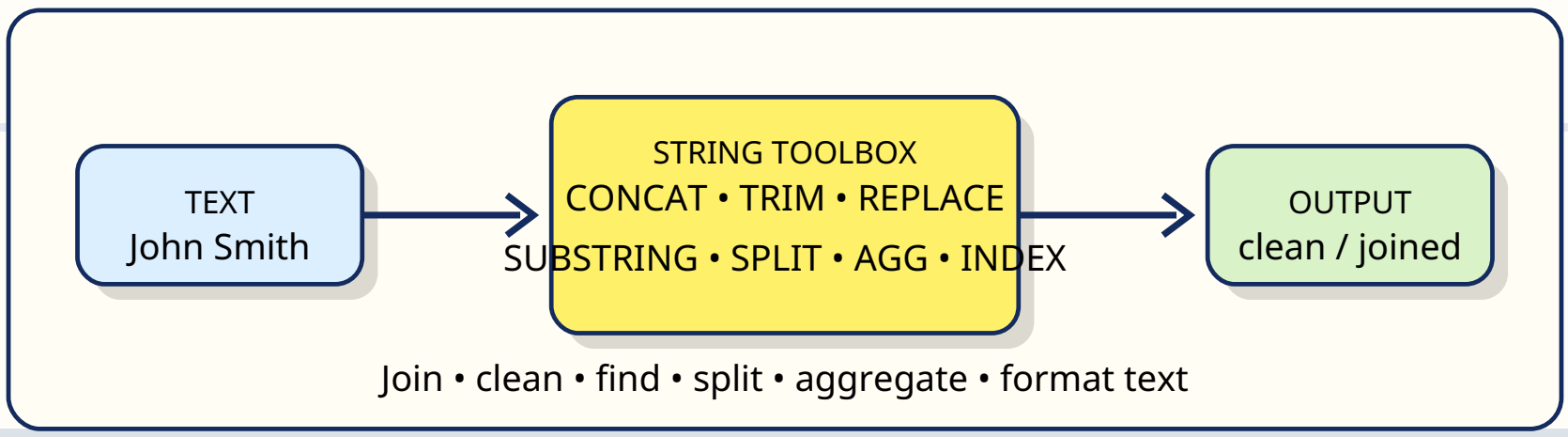
Numeric Functions

NULL Handling Functions

Conditional Expressions

Conversion Functions
```

STRING FUNCTIONS



String functions work with text values.

Important functions include:

EXAMPLE / FLOW

```
CONCAT
CONCAT_WS
LEFT
RIGHT
LEN
DATALENGTH
TRIM
LTRIM
RTRIM
UPPER
LOWER
REPLACE
SUBSTRING
STRING_SPLIT
STRING_AGG
CHARINDEX
PATINDEX
STUFF
REVERSE
REPLICATE
SPACE
FORMAT
ASCII
CHAR
NCHAR
UNICODE
```

CONCAT

Simple Definition

`CONCAT` joins multiple values into one string.

Syntax

SQL

```sql
CONCAT(value1, value2, ...)
```

Example

SQL

```sql
SELECT CONCAT('John', ' ', 'Smith');
```

Result:

EXAMPLE / FLOW

```
John Smith
```

With columns:

SQL

```sql
SELECT
    CONCAT(FirstName, ' ', LastName) AS FullName
FROM Employees;
```

Important NULL Behavior

`CONCAT` treats `NULL` as an empty string.

Example:

```sql
SELECT CONCAT('John', NULL, 'Smith');
```

Result:

```
JohnSmith
```

This is different from using +.

For example:

```sql
SELECT 'John' + NULL + 'Smith';
```

typically returns:

```
NULL
```

under normal SQL Server NULL behavior.

When to Use

Use `CONCAT` when combining:

- First and last names
- Address components
- Display values
- Text and numbers
- Multiple nullable columns

CONCAT_WS

`CONCAT_WS` means:

Concatenate With Separator

It joins multiple values while automatically placing a separator between them.

Syntax

```sql
CONCAT_WS(separator, value1, value2, ...)
```

Example:

SQL

```
SELECT CONCAT_WS(
    ', ',
    'New York',
    'NY',
    'USA'
);
```

Result:

EXAMPLE / FLOW

```
New York, NY, USA
```

Another example:

SQL

```
SELECT
    CONCAT_WS(
        ' ',
        FirstName,
        MiddleName,
        LastName
    ) AS FullName
FROM Employees;
```

One useful feature is that `CONCAT_WS` skips `NULL` values rather than creating unnecessary separators around them.

When to Use

Very useful for:

EXAMPLE / FLOW

```
Full names
Addresses
CSV-style display
Combined labels
```

LEFT

`LEFT` returns characters from the beginning of a string.

Syntax

SQL

```
LEFT(expression, number_of_characters)
```

Example:

SQL

```
SELECT LEFT('SQL Server', 3);
```

Result:

EXAMPLE / FLOW

```
SQL
```

Example:

```sql
SELECT LEFT(EmployeeCode, 3)
FROM Employees;
```

Use when you need a fixed number of characters from the left side.

RIGHT

`RIGHT` returns characters from the end of a string.

Syntax

```sql
RIGHT(expression, number_of_characters)
```

Example:

```sql
SELECT RIGHT('ABC12345', 4);
```

Result:

```
2345
```

Common use:

```
Last four digits
File extensions
Code suffixes
```

LEN

`LEN` returns the number of characters in a string.

Example:

```sql
SELECT LEN('SQL Server');
```

Result:

```
10
```

Important:

LEN normally ignores trailing spaces.

Example:

```sql
SELECT LEN('ABC   ');
```

Result:

```
EXAMPLE / FLOW
3
```

because trailing spaces aren't counted.

DATALENGTH

DATALENGTH returns the number of **bytes** used by a value.

Example:

```sql
SELECT DATALENGTH('ABC');
```

Result:

```
EXAMPLE / FLOW
3
```

For Unicode data, byte size can differ.

Example:

```sql
SELECT DATALENGTH(N'ABC');
```

can return:

```
EXAMPLE / FLOW
6
```

because NVARCHAR traditionally uses two bytes per UTF-16 code unit.

LEN vs DATALENGTH

LENDATALENGTH	
Returns character count	Returns byte count
Ignores trailing spaces	Includes storage bytes
Useful for text length	Useful for storage size

Memory:

EXAMPLE / FLOW

```
LEN
= How many characters?

DATALENGTH
= How many bytes?
```

TRIM

`TRIM` removes unwanted spaces from both sides of a string.

Example:

SQL

```
SELECT TRIM('   John   ');
```

Result:

EXAMPLE / FLOW

```
John
```

Use for cleaning imported/user-entered text.

Modern SQL Server also supports more advanced trimming options, but the basic and most common use is removing leading and trailing spaces.

LTRIM

`LTRIM` removes spaces from the **left** side.

SQL

```
SELECT LTRIM('   John');
```

Result:

EXAMPLE / FLOW

```
John
```

RTRIM

`RTRIM` removes spaces from the **right** side.

SQL

```
SELECT RTRIM('John   ');
```

Result:

EXAMPLE / FLOW

```
John
```

Memory:

EXAMPLE / FLOW

```
LTRIM
→ Left spaces

RTRIM
→ Right spaces

TRIM
→ Both sides
```

UPPER

Converts characters to uppercase.

SQL

```sql
SELECT UPPER('Sql Server');
```

Result:

EXAMPLE / FLOW

```
SQL SERVER
```

Common use:

EXAMPLE / FLOW

```
Display formatting
Normalizing comparison values
Standardized codes
```

LOWER

Converts characters to lowercase.

SQL

```sql
SELECT LOWER('SQL SERVER');
```

Result:

EXAMPLE / FLOW

```
sql server
```

Useful for display or normalization.

Remember that whether normal string comparisons are case-sensitive is determined primarily by collation, not by `UPPER`/`LOWER`.

REPLACE

REPLACE replaces every occurrence of one string with another.

Syntax

SQL

```
REPLACE(expression, search, replacement)
```

Example:

SQL

```
SELECT REPLACE(
    'SQL Server 2022',
    '2022',
    '2025'
);
```

Result:

EXAMPLE / FLOW

```
SQL Server 2025
```

Another:

SQL

```
SELECT REPLACE(
    '555-123-4567',
    '-',
    ''
);
```

Result:

EXAMPLE / FLOW

```
5551234567
```

Useful for:

EXAMPLE / FLOW

```
Cleaning text
Removing unwanted characters
Replacing known values
Formatting
```

SUBSTRING

SUBSTRING extracts part of a string.

Syntax

SQL

```
SUBSTRING(expression, start, length)
```

Important:

SQL Server character positions start at 1.

Example:

```sql
SELECT SUBSTRING(
    'SQL Server',
    1,
    3
);
```

Result:

EXAMPLE / FLOW

```
SQL
```

Another:

```sql
SELECT SUBSTRING(
    'EMP-10025',
    5,
    5
);
```

Result:

EXAMPLE / FLOW

```
10025
```

STRING_SPLIT

`STRING_SPLIT` converts a delimited string into multiple rows.

Suppose:

EXAMPLE / FLOW

```
IT,HR,Finance
```

Execute:

```sql
SELECT value
FROM STRING_SPLIT(
    'IT,HR,Finance',
    ','
);
```

Result conceptually:

EXAMPLE / FLOW

```
value
-------
IT
HR
Finance
```

When to Use

Useful for:

EXAMPLE / FLOW

```
Delimited input
Tag processing
Simple comma-separated lists
```

Important Design Point

Do not use comma-separated strings as a replacement for proper relational design.

For example, storing:

EXAMPLE / FLOW

```
CustomerIds = '1,2,3,4'
```

inside one column is usually poor relational design.

`STRING_SPLIT` is useful when delimited data genuinely arrives from an external source or parameter.

STRING_AGG

`STRING_AGG` performs the opposite kind of operation.

It combines values from multiple rows into one string.

Example data:

EXAMPLE / FLOW

```
Department
----------
IT
HR
Finance
```

Query:

SQL

```sql
SELECT STRING_AGG(
    DepartmentName,
    ', '
)
FROM Departments;
```

Result:

EXAMPLE / FLOW

```
IT, HR, Finance
```

When to Use

Useful for:

EXAMPLE / FLOW

```
Comma-separated reports
Display lists
Combining related values
Export output
```

STRING_SPLIT vs STRING_AGG

EXAMPLE / FLOW

```
STRING_SPLIT
One string
→ Multiple rows
```

EXAMPLE / FLOW

```
STRING_AGG
Multiple rows
→ One string
```

CHARINDEX

`CHARINDEX` finds the position of one string inside another.

Syntax

SQL

```sql
CHARINDEX(search_expression, expression)
```

Example:

SQL

```sql
SELECT CHARINDEX(
    'Server',
    'SQL Server Database'
);
```

Result:

EXAMPLE / FLOW

```
5
```

Because `Server` starts at character 5.

If not found:

SQL

```sql
SELECT CHARINDEX(
    'Oracle',
    'SQL Server'
);
```

returns:

EXAMPLE / FLOW

```
0
```

PATINDEX

`PATINDEX` searches for a pattern and returns its starting position.

Unlike `CHARINDEX`, it supports wildcard patterns.

Example:

SQL

```
SELECT PATINDEX(
    '%Server%',
    'SQL Server Database'
);
```

Result:

EXAMPLE / FLOW

```
5
```

Another:

SQL

```
SELECT PATINDEX(
    '%[0-9]%',
    'ABC123'
);
```

returns the position of the first digit.

CHARINDEX vs PATINDEX

CHARINDEXPATINDEX	
Searches literal string	Searches pattern
No wildcard required	Supports `%`, `_`, ranges
Easier exact text search	Useful pattern detection

Memory:

EXAMPLE / FLOW

```
CHARINDEX
→ Find text

PATINDEX
→ Find pattern
```

STUFF

STUFF deletes part of a string and inserts another string at that position.

Syntax

SQL

```
STUFF(
    expression,
    start,
    length,
    replacement
)
```

Example:

SQL

```
SELECT STUFF(
    'SQL 2022 Server',
    5,
    4,
    '2025'
);
```

Result:

EXAMPLE / FLOW

```
SQL 2025 Server
```

Another:

SQL

```
SELECT STUFF(
    '1234567890',
    4,
    3,
    'XXX'
);
```

Result:

EXAMPLE / FLOW

```
123XXX7890
```

REVERSE

Reverses a string.

SQL

```
SELECT REVERSE('SQL');
```

Result:

EXAMPLE / FLOW

```
LQS
```

Useful occasionally for:

EXAMPLE / FLOW

```
Reverse searching
String manipulation
Certain parsing problems
```

REPLICATE

Repeats a string a specified number of times.

Syntax

SQL

```
REPLICATE(expression, count)
```

Example:

SQL

```
SELECT REPLICATE('*', 5);
```

Result:

EXAMPLE / FLOW

```
*****
```

Useful for:

EXAMPLE / FLOW

```
Masking
Padding
Display formatting
```

Example:

SQL

```
SELECT
    REPLICATE('*', 8)
    + RIGHT(CardNumber, 4);
```

SPACE

`SPACE(n)` returns a string containing `n` spaces.

Example:

SQL

```
SELECT
    'John'
    + SPACE(5)
    + 'Smith';
```

Result conceptually:

EXAMPLE / FLOW

```
John     Smith
```

Mostly useful for formatting fixed-layout text.

FORMAT

`FORMAT` converts numbers or dates into formatted strings.

Example:

SQL

```
SELECT FORMAT(
    12345.67,
    'N2'
);
```

Possible result:

EXAMPLE / FLOW

```
12,345.67
```

Currency:

SQL

```
SELECT FORMAT(
    12345.67,
    'C',
    'en-US'
);
```

Possible result:

EXAMPLE / FLOW

```
$12,345.67
```

Date:

SQL

```
SELECT FORMAT(
    GETDATE(),
    'yyyy-MM-dd'
);
```

When to Use FORMAT

Useful for:

EXAMPLE / FLOW

```
Reports
User-facing display
Culture-specific formatting
```

When Not to Use FORMAT

Avoid using `FORMAT` heavily in large performance-sensitive queries.

It is generally more expensive than simpler SQL Server conversion functions.

Also remember:

Formatting usually belongs in the presentation layer of an application when possible.

For an Angular/.NET application, date and currency display is often better handled by the application/UI.

ASCII

Returns the ASCII code of the first character.

Example:

SQL

```sql
SELECT ASCII('A');
```

Result:

EXAMPLE / FLOW

```
65
```

CHAR

Converts an integer ASCII code into a character.

SQL

```sql
SELECT CHAR(65);
```

Result:

EXAMPLE / FLOW

```
A
```

Thus:

EXAMPLE / FLOW

```
ASCII('A')
→ 65

CHAR(65)
→ A
```

UNICODE

Returns the Unicode code point/value associated with the first character.

Example:

SQL

```
SELECT UNICODE(N'A');
```

Result:

EXAMPLE / FLOW

```
65
```

NCHAR

Returns the Unicode character represented by an integer.

Example:

SQL

```
SELECT NCHAR(65);
```

Result:

EXAMPLE / FLOW

```
A
```

Memory:

EXAMPLE / FLOW

```
ASCII
Character → Number

CHAR
Number → Character
```

EXAMPLE / FLOW

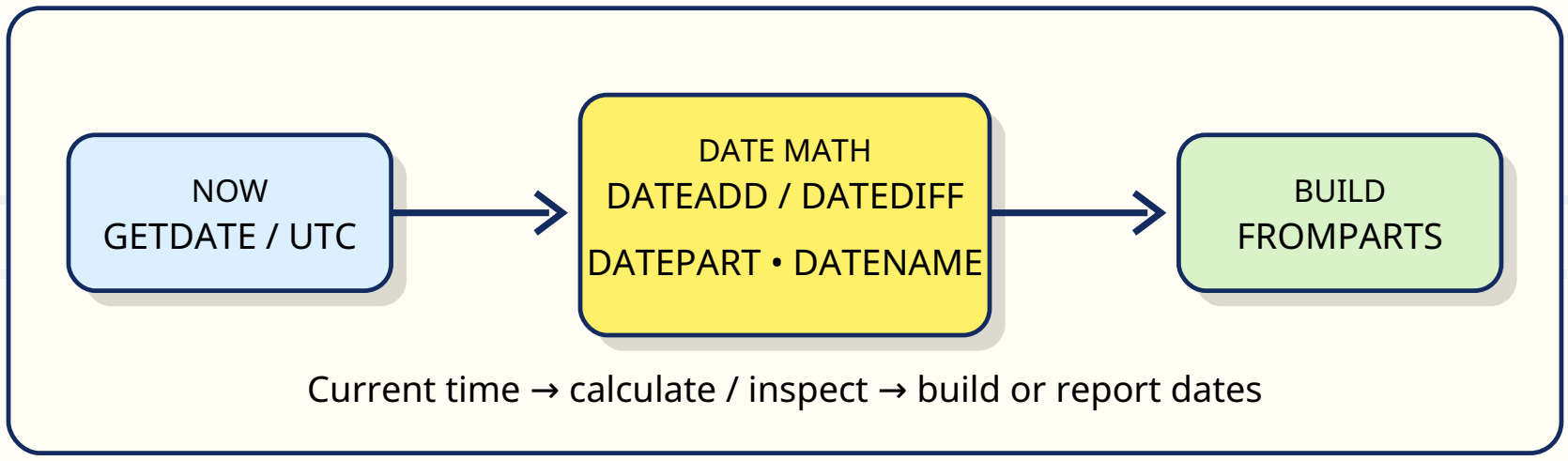
```
UNICODE
Unicode character → Number

NCHAR
Number → Unicode character
```

String Function Cheat Sheet

FunctionPurpose	
CONCAT	Join strings
CONCAT_WS	Join with separator
LEFT	Characters from left
RIGHT	Characters from right
LEN	Character length
DATALENGTH	Byte length
TRIM	Remove spaces both sides
LTRIM	Remove left spaces
RTRIM	Remove right spaces
UPPER	Uppercase
LOWER	Lowercase
REPLACE	Replace text
SUBSTRING	Extract part
STRING_SPLIT	String → rows
STRING_AGG	Rows → string
CHARINDEX	Find text position
PATINDEX	Find pattern position
STUFF	Delete/insert text
REVERSE	Reverse text
REPLICATE	Repeat text
SPACE	Generate spaces
FORMAT	Format display value
ASCII	Character → ASCII number
CHAR	ASCII number → character
UNICODE	Unicode character → number
NCHAR	Number → Unicode character

DATE AND TIME FUNCTIONS



Important SQL Server date/time functions include:

EXAMPLE / FLOW

```
GETDATE
GETUTCDATE
SYSDATETIME
SYSUTCDATETIME
SYSDATETIMEOFFSET
CURRENT_TIMESTAMP

DATEADD
DATEDIFF
DATEDIFF_BIG
DATEPART
DATENAME
YEAR
MONTH
DAY
EOMONTH
DATEFROMPARTS
TIMEFROMPARTS
DATETIMEFROMPARTS
DATETIME2FROMPARTS
```

GETDATE

Returns the current SQL Server system date and time as `DATETIME`.

SQL

```
SELECT GETDATE();
```

Example result:

EXAMPLE / FLOW

```
2026-09-14 15:25:30.123
```

Use when local server date/time is required.

GETUTCDATE

Returns current UTC date/time as `DATETIME`.

SQL

```
SELECT GETUTCDATE();
```

Conceptually:

EXAMPLE / FLOW

```
GETDATE()
→ Server/local time

GETUTCDATE()
→ UTC
```

SYSDATETIME

Returns current system date/time using the more precise `DATETIME2` type.

SQL

```
SELECT SYSDATETIME();
```

It provides greater fractional-second precision than `GETDATE()`.

SYSUTCDATETIME

Returns current UTC date/time as `DATETIME2`.

SQL

```
SELECT SYSUTCDATETIME();
```

For modern applications storing UTC timestamps, this is very useful.

Example:

SQL

```
CreatedAt DATETIME2(3)
    DEFAULT SYSUTCDATETIME()
```

SYSDATETIMEOFFSET

Returns:

EXAMPLE / FLOW

```
Date
+
Time
+
UTC offset
```

Example:

```sql
SELECT SYSDATETIMEOFFSET();
```

Possible value:

```
2026-09-14 15:30:00.1234567 -04:00
```

CURRENT_TIMESTAMP

Returns current date/time.

```sql
SELECT CURRENT_TIMESTAMP;
```

Notice:

```
CURRENT_TIMESTAMP
```

does not require parentheses.

In SQL Server it is functionally similar to:

```sql
GETDATE()
```

for current local/server datetime behavior.

Current Date/Time Function Cheat Sheet

FunctionReturns	
`GETDATE()`	Current local/server `DATETIME`
`GETUTCDATE()`	Current UTC `DATETIME`
`SYSDATETIME()`	Current local `DATETIME2`
`SYSUTCDATETIME()`	Current UTC `DATETIME2`
`SYSDATETIMEOFFSET()`	Current date/time + offset
`CURRENT_TIMESTAMP`	Current date/time

For most modern UTC-based systems:

```
SYSUTCDATETIME()
```

is often a good choice.

DATEADD

Adds or subtracts time from a date.

Syntax

SQL

```
DATEADD(datepart, number, date)
```

Add 10 days:

SQL

```
SELECT DATEADD(
    DAY,
    10,
    '2026-09-01'
);
```

Result:

EXAMPLE / FLOW

```
2026-09-11
```

Add three months:

SQL

```
SELECT DATEADD(
    MONTH,
    3,
    '2026-01-15'
);
```

Subtract seven days:

SQL

```
SELECT DATEADD(
    DAY,
    -7,
    GETDATE()
);
```

Common Date Parts

EXAMPLE / FLOW

```
YEAR
QUARTER
MONTH
DAY
WEEK
HOUR
MINUTE
SECOND
MILLISECOND
```

DATEDIFF

Returns the number of specified date-part boundaries between two dates.

Syntax

SQL

```
DATEDIFF(
    datepart,
    startdate,
    enddate
)
```

Example:

SQL

```
SELECT DATEDIFF(
    DAY,
    '2026-09-01',
    '2026-09-10'
);
```

Result:

EXAMPLE / FLOW

```
9
```

Years:

SQL

```
SELECT DATEDIFF(
    YEAR,
    '2020-01-01',
    '2026-01-01'
);
```

Result:

EXAMPLE / FLOW

```
6
```

Important DATEDIFF Rule

`DATEDIFF` counts **date-part boundaries**, not always the exact elapsed duration you might intuitively expect.

Therefore, calculating a person's age simply as:

SQL

```
DATEDIFF(YEAR, BirthDate, GETDATE())
```

can be wrong before the birthday occurs that year.

Age calculations need additional logic when exact age matters.

DATEDIFF_BIG

Works similarly to `DATEDIFF` but returns a:

EXAMPLE / FLOW

```
BIGINT
```

instead of:

EXAMPLE / FLOW

```
INT
```

Use when the result could exceed `INT` range.

Example:

SQL

```
SELECT DATEDIFF_BIG(
    MILLISECOND,
    '2000-01-01',
    GETDATE()
);
```

DATEPART

Returns a numeric component from a date.

Example:

SQL

```
SELECT DATEPART(
    MONTH,
    '2026-09-14'
);
```

Result:

EXAMPLE / FLOW

```
9
```

Other examples:

SQL

```
SELECT DATEPART(
    YEAR,
    GETDATE()
);
```

SQL

```
SELECT DATEPART(
    HOUR,
    SYSDATETIME()
);
```

DATENAME

Returns a textual name/representation of a date part.

Example:

SQL

```
SELECT DATENAME(
    MONTH,
    '2026-09-14'
);
```

Result:

EXAMPLE / FLOW

```
September
```

Example:

SQL

```
SELECT DATENAME(
    WEEKDAY,
    '2026-09-14'
);
```

returns the weekday name according to SQL Server language/session settings.

DATEPART vs DATENAME

EXAMPLE / FLOW

```
DATEPART
→ Number

DATENAME
→ Text
```

Example:

SQL

```
DATEPART(MONTH, '2026-09-14')
→ 9
```

SQL

```
DATENAME(MONTH, '2026-09-14')
→ September
```

YEAR

Returns the year component.

SQL

```
SELECT YEAR('2026-09-14');
```

Result:

EXAMPLE / FLOW

```
2026
```

MONTH

Returns month number.

SQL

```
SELECT MONTH('2026-09-14');
```

Result:

EXAMPLE / FLOW

```
9
```

DAY

Returns day of month.

SQL

```
SELECT DAY('2026-09-14');
```

Result:

EXAMPLE / FLOW

```
14
```

These are convenient shortcuts for common `DATEPART` operations.

EOMONTH

Returns the final date of a month.

Example:

SQL

```
SELECT EOMONTH('2026-02-15');
```

Result:

EXAMPLE / FLOW

```
2026-02-28
```

Another:

SQL

```
SELECT EOMONTH('2024-02-15');
```

Result:

EXAMPLE / FLOW

```
2024-02-29
```

because 2024 was a leap year.

EOMONTH with Offset

You can request another month's ending date.

SQL

```
SELECT EOMONTH(
    '2026-09-14',
    1
);
```

Returns the end of the following month.

Useful for:

EXAMPLE / FLOW

```
Billing periods
Month-end reporting
Financial calculations
```

DATEFROMPARTS

Builds a valid `DATE` from year, month, and day values.

Syntax

SQL

```
DATEFROMPARTS(
    year,
    month,
    day
)
```

Example:

SQL

```
SELECT DATEFROMPARTS(
    2026,
    9,
    14
);
```

Result:

EXAMPLE / FLOW

```
2026-09-14
```

This is better than manually concatenating strings when you already have numeric date parts.

TIMEFROMPARTS

Constructs a `TIME` value.

Example:

SQL

```
SELECT TIMEFROMPARTS(
    14,
    30,
    0,
    0,
    0
);
```

Represents:

EXAMPLE / FLOW

```
14:30:00
```

Its complete syntax includes precision and fractional seconds.

DATETIMEFROMPARTS

Constructs a legacy `DATETIME` value from components.

Example:

SQL

```
SELECT DATETIMEFROMPARTS(
    2026,
    9,
    14,
    14,
    30,
    0,
    0
);
```

DATETIME2FROMPARTS

For modern `DATETIME2` values:

```sql
SELECT DATETIME2FROMPARTS(
    2026,
    9,
    14,
    14,
    30,
    0,
    123,
    3
);
```

Use when individual year/month/day/time components must be combined into a proper `DATETIME2`.

Date Calculations

Add 30 Days

```sql
SELECT DATEADD(
    DAY,
    30,
    GETDATE()
);
```

Previous 7 Days

```sql
SELECT DATEADD(
    DAY,
    -7,
    GETDATE()
);
```

Difference in Days

```sql
SELECT DATEDIFF(
    DAY,
    '2026-09-01',
    '2026-09-14'
);
```

First Day of Current Month

One approach:

```sql
SELECT DATEFROMPARTS(
    YEAR(GETDATE()),
    MONTH(GETDATE()),
    1
);
```

Last Day of Current Month

```sql
SELECT EOMONTH(GETDATE());
```

Date Filtering Performance Rule

Avoid doing this unnecessarily:

```sql
WHERE YEAR(OrderDate) = 2026;
```

For large indexed tables, a range is often more efficient:

```sql
WHERE OrderDate >= '2026-01-01'
  AND OrderDate <  '2027-01-01';
```

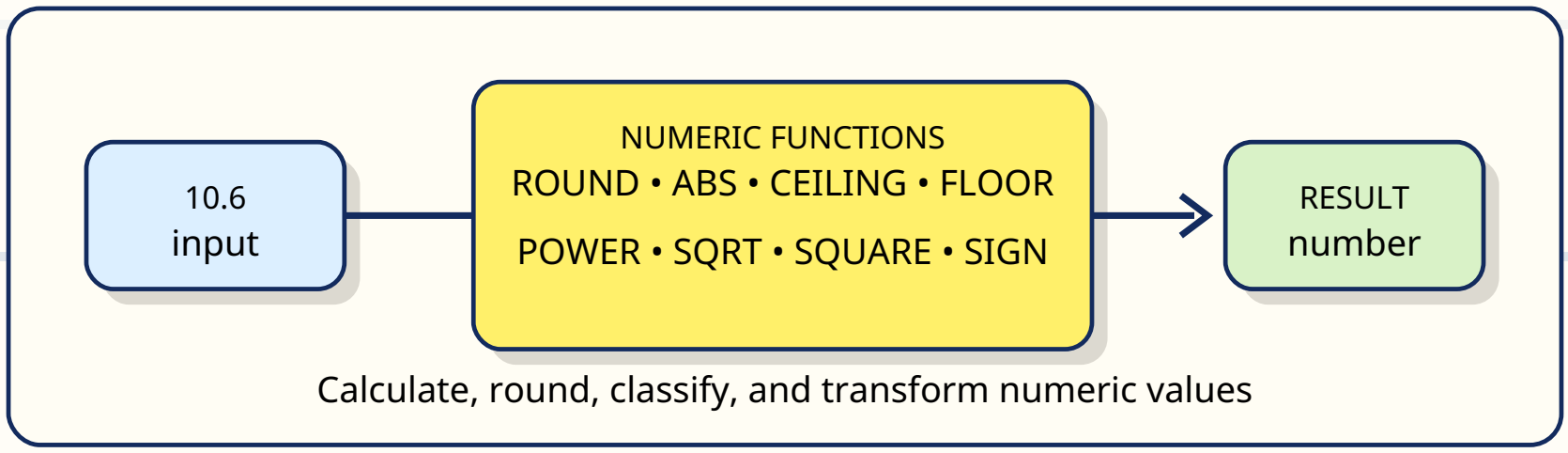
The second form can make it easier for SQL Server to use an index efficiently.

Date/Time Cheat Sheet

FunctionPurpose	
GETDATE	Current local datetime
GETUTCDATE	Current UTC datetime
SYSDATETIME	Precise local datetime
SYSUTCDATETIME	Precise UTC datetime
SYSDATETIMEOFFSET	Datetime + offset
CURRENT_TIMESTAMP	Current datetime
DATEADD	Add/subtract date units
DATEDIFF	Difference
DATEDIFF_BIG	Large difference
DATEPART	Numeric date part
DATENAME	Text date part
YEAR	Year
MONTH	Month
DAY	Day
EOMONTH	Last day of month
DATEFROMPARTS	Build DATE
TIMEFROMPARTS	Build TIME

FunctionPurpose	
DATETIMEFROMPARTS	Build DATETIME
DATETIME2FROMPARTS	Build DATETIME2

NUMERIC FUNCTIONS



Important numeric functions:

EXAMPLE / FLOW

```
ROUND
ABS
CEILING
FLOOR
POWER
SQRT
SIGN
RAND
PI
SQUARE
```

ROUND

Rounds a numeric value.

Syntax

SQL

```
ROUND(number, decimal_places)
```

Example:

SQL

```
SELECT ROUND(
    123.4567,
    2
);
```

Result:

EXAMPLE / FLOW

```
123.4600
```

depending on input data type/scale.

Round to zero decimals:

SQL

```
SELECT ROUND(
    123.56,
    0
);
```

Result:

EXAMPLE / FLOW

```
124
```

ROUND with Negative Length

You can round digits to the left of the decimal.

SQL

```
SELECT ROUND(
    12345,
    -2
);
```

Result:

EXAMPLE / FLOW

```
12300
```

Useful for:

EXAMPLE / FLOW

```
Financial reporting
Summary values
Display calculations
```

ABS

Returns the absolute value.

SQL

```
SELECT ABS(-100);
```

Result:

EXAMPLE / FLOW

```
100
```

Another:

SQL

```
SELECT ABS(50);
```

Result:

EXAMPLE / FLOW

```
50
```

It removes the sign mathematically.

Useful for distance/difference calculations.

CEILING

Returns the smallest integer greater than or equal to a number.

SQL

```
SELECT CEILING(10.1);
```

Result:

EXAMPLE / FLOW

```
11
```

SQL

```
SELECT CEILING(10.9);
```

Result:

EXAMPLE / FLOW

```
11
```

Think:

EXAMPLE / FLOW

```
CEILING
→ Go upward
```

FLOOR

Returns the largest integer less than or equal to a number.

SQL

```
SELECT FLOOR(10.9);
```

Result:

EXAMPLE / FLOW

```
10
```

SQL

```
SELECT FLOOR(10.1);
```

Result:

EXAMPLE / FLOW

```
10
```

Think:

EXAMPLE / FLOW

```
FLOOR
→ Go downward
```

ROUND vs CEILING vs FLOOR

For:

EXAMPLE / FLOW

```
10.6
```

EXAMPLE / FLOW

```
ROUND
→ 11

CEILING
→ 11

FLOOR
→ 10
```

For:

EXAMPLE / FLOW

```
10.2
```

EXAMPLE / FLOW

```
ROUND
→ 10

CEILING
→ 11

FLOOR
→ 10
```

POWER

Raises a number to a power.

SQL

```
SELECT POWER(
    2,
    3
);
```

Result:

EXAMPLE / FLOW

```
8
```

because:

EXAMPLE / FLOW

```
2 × 2 × 2 = 8
```

SQRT

Returns the square root.

SQL

```
SELECT SQRT(25);
```

Result:

EXAMPLE / FLOW

```
5
```

SQUARE

Returns a value multiplied by itself.

SQL

```
SELECT SQUARE(5);
```

Result:

EXAMPLE / FLOW

```
25
```

Equivalent conceptually to:

SQL

```
POWER(5, 2)
```

SIGN

Returns whether a number is positive, zero, or negative.

SQL

```
SELECT SIGN(-100);
```

Result:

EXAMPLE / FLOW

```
-1
```

SQL

```
SELECT SIGN(0);
```

Result:

EXAMPLE / FLOW

```
0
```

SQL

```
SELECT SIGN(100);
```

Result:

EXAMPLE / FLOW

```
1
```

Useful when categorizing values by direction.

RAND

Generates an approximate pseudo-random number between:

EXAMPLE / FLOW

```
0
and
1
```

Example:

SQL

```
SELECT RAND();
```

Possible result:

EXAMPLE / FLOW

```
0.583245...
```

Generate approximately 1 through 100:

SQL

```
SELECT FLOOR(
    RAND() * 100
) + 1;
```

Do not treat `RAND()` as a cryptographically secure random generator.

PI

Returns the mathematical value π.

SQL

```
SELECT PI();
```

Result approximately:

EXAMPLE / FLOW

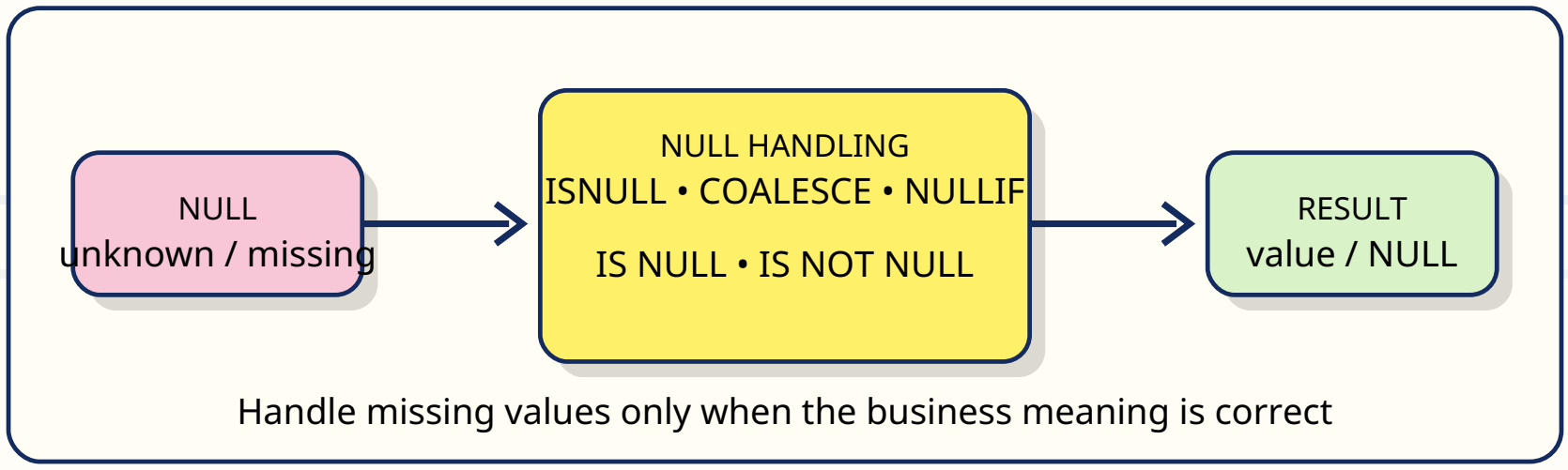
```
3.14159265358979
```

Useful in mathematical calculations.

Numeric Cheat Sheet

FunctionPurpose	
`ROUND`	Round number
`ABS`	Absolute value
`CEILING`	Round upward
`FLOOR`	Round downward
`POWER`	Raise to power
`SQRT`	Square root
`SQUARE`	Number squared
`SIGN`	Positive/negative/zero
`RAND`	Pseudo-random value
`PI`	π

NULL HANDLING



NULL handling is extremely important in SQL Server.

Remember:

EXAMPLE / FLOW

```
NULL
≠ 0

NULL
≠ empty string

NULL
≠ false
```

NULL means:

EXAMPLE / FLOW

```
Unknown
Missing
Not available
Not applicable
```

NULL Comparison

Wrong:

SQL

```sql
WHERE City = NULL
```

Correct:

SQL

```sql
WHERE City IS NULL;
```

For rows that have a value:

SQL

```sql
WHERE City IS NOT NULL;
```

ISNULL

`ISNULL` replaces a NULL value with another value.

Syntax

SQL

```
ISNULL(expression, replacement)
```

Example:

SQL

```
SELECT ISNULL(
    MiddleName,
    'Not Provided'
)
FROM Customers;
```

If:

EXAMPLE / FLOW

```
MiddleName = NULL
```

result becomes:

EXAMPLE / FLOW

```
Not Provided
```

Another:

SQL

```
SELECT ISNULL(Bonus, 0)
FROM Employees;
```

COALESCE

`COALESCE` returns the first non-NULL expression from a list.

Example:

SQL

```
SELECT COALESCE(
    MobilePhone,
    HomePhone,
    OfficePhone,
    'No Phone'
)
FROM Customers;
```

SQL Server checks:

EXAMPLE / FLOW

```
MobilePhone
↓
If NULL

HomePhone
↓
If NULL

OfficePhone
↓
If NULL

'No Phone'
```

The first non-NULL value is returned.

ISNULL vs COALESCE

Simple difference:

EXAMPLE / FLOW

```
ISNULL
→ Two arguments

COALESCE
→ Multiple arguments
```

Example:

SQL

```sql
ISNULL(Phone, 'Unknown')
```

versus:

SQL

```sql
COALESCE(
    Mobile,
    HomePhone,
    OfficePhone,
    'Unknown'
)
```

There are also important differences in returned data type and nullability metadata, so they are not always interchangeable.

For beginner learning:

> Use `ISNULL` for a simple SQL Server-specific two-value replacement; use `COALESCE` when you need the first non-NULL value from multiple possibilities.

NULLIF

`NULLIF` compares two expressions.

If they are equal:

EXAMPLE / FLOW

```
Returns NULL
```

If they differ:

EXAMPLE / FLOW

```
Returns first expression
```

Example:

SQL

```
SELECT NULLIF(
    100,
    100
);
```

Result:

EXAMPLE / FLOW

```
NULL
```

Example:

SQL

```
SELECT NULLIF(
    100,
    50
);
```

Result:

EXAMPLE / FLOW

```
100
```

Useful NULLIF Example — Prevent Divide by Zero

This can fail:

SQL

```
SELECT TotalSales / NumberOfCustomers
FROM Sales;
```

if:

EXAMPLE / FLOW

```
NumberOfCustomers = 0
```

Instead:

SQL

```
SELECT
    TotalSales /
    NULLIF(NumberOfCustomers, 0)
FROM Sales;
```

If denominator is zero:

EXAMPLE / FLOW

```
NULLIF(0,0)
→ NULL
```

Then division returns `NULL` rather than dividing by zero.

This is a very useful SQL pattern.

NULL in Calculations

NULL normally propagates through ordinary arithmetic.

Example:

SQL

```
SELECT 100 + NULL;
```

Result:

EXAMPLE / FLOW

```
NULL
```

Similarly:

EXAMPLE / FLOW

```
100 * NULL
→ NULL
```

If you want NULL treated as zero:

SQL

```
SELECT
    100 + ISNULL(Bonus, 0);
```

But only do this if zero is the correct business meaning.

Do not automatically convert every NULL to zero.

NULL in Aggregate Functions

Most common aggregate functions ignore NULL values.

Suppose:

EXAMPLE / FLOW

```
Salary
------
50000
60000
NULL
70000
```

Then:

SQL

```
SELECT AVG(Salary)
FROM Employees;
```

uses:

EXAMPLE / FLOW

```
50000
60000
70000
```

and ignores the NULL.

COUNT and NULL

This:

SQL

```
COUNT(*)
```

counts rows.

This:

SQL

```
COUNT(Salary)
```

counts only rows where:

EXAMPLE / FLOW

```
Salary IS NOT NULL
```

Example:

EXAMPLE / FLOW

```
4 rows total
1 Salary NULL
```

Then:

EXAMPLE / FLOW

```
COUNT(*)
→ 4

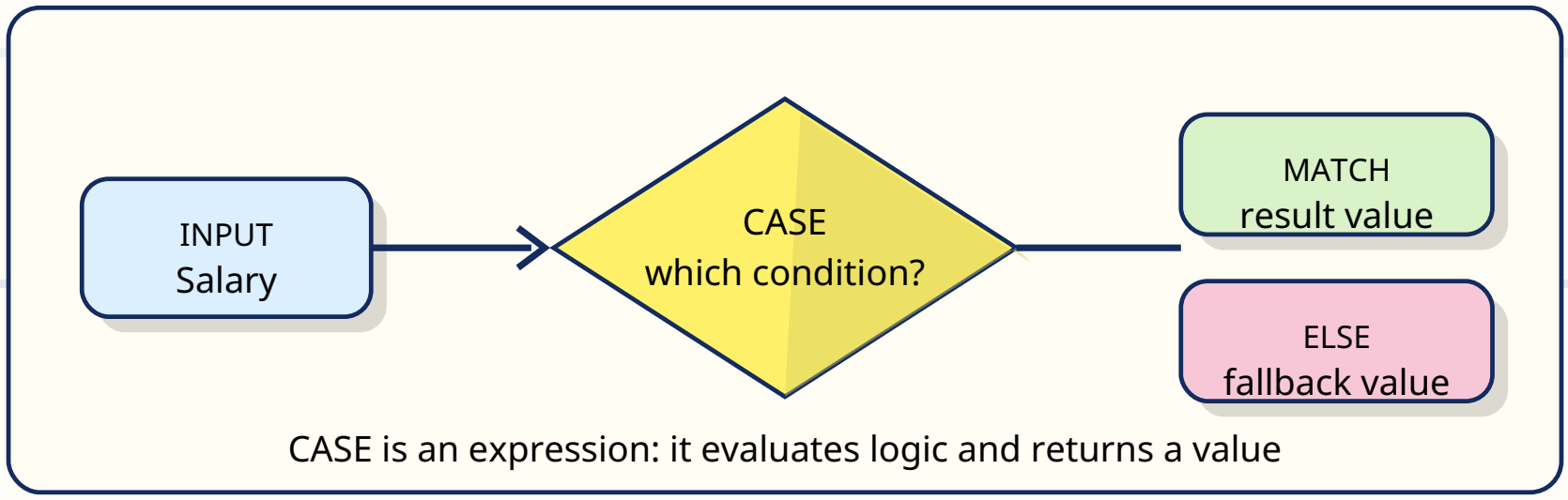
COUNT(Salary)
→ 3
```

This is an important interview question.

NULL Cheat Sheet

ExpressionPurpose	
`IS NULL`	Find NULL
`IS NOT NULL`	Find non-NULL
`ISNULL(a,b)`	Replace NULL
`COALESCE(a,b,c)`	First non-NULL
`NULLIF(a,b)`	NULL when equal
`COUNT(*)`	Counts rows
`COUNT(column)`	Ignores NULL

CONDITIONAL EXPRESSIONS — CASE



CASE is an expression: it evaluates logic and returns a value

`CASE` allows conditional logic inside SQL expressions.

Think:

EXAMPLE / FLOW

```
IF condition
THEN result
ELSE another result
```

SQL Server provides two common forms:

EXAMPLE / FLOW

```
Simple CASE

Searched CASE
```

Simple CASE

Used when comparing one expression against several possible values.

Syntax

SQL

```
CASE expression
    WHEN value1 THEN result1
    WHEN value2 THEN result2
    ELSE result
END
```

Example:

SQL

```
SELECT
    EmployeeName,

    CASE DepartmentId
        WHEN 1 THEN 'IT'
        WHEN 2 THEN 'HR'
        WHEN 3 THEN 'Finance'
        ELSE 'Other'
    END AS DepartmentName

FROM Employees;
```

Here SQL Server compares:

EXAMPLE / FLOW

```
DepartmentId
```

against:

EXAMPLE / FLOW

```
1
2
3
```

Searched CASE

Used when each branch contains its own condition.

Syntax

SQL

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    ELSE result
END
```

Example:

```sql
SELECT
    EmployeeName,
    Salary,

    CASE
        WHEN Salary >= 100000 THEN 'High'
        WHEN Salary >= 60000 THEN 'Medium'
        ELSE 'Low'
    END AS SalaryLevel

FROM Employees;
```

Simple CASE vs Searched CASE

Simple:

```sql
CASE Status
    WHEN 1 THEN 'Active'
    WHEN 2 THEN 'Inactive'
END
```

Searched:

```sql
CASE
    WHEN Salary >= 100000 THEN 'High'
    WHEN Salary >= 50000 THEN 'Medium'
    ELSE 'Low'
END
```

Memory:

EXAMPLE / FLOW

```
Simple CASE
→ Compare one expression

Searched CASE
→ Evaluate different conditions
```

Conditional Labeling

CASE is very useful for creating display labels.

Example:

```sql
SELECT
    OrderId,

    CASE
        WHEN TotalAmount >= 10000
            THEN 'Large Order'

        WHEN TotalAmount >= 1000
            THEN 'Medium Order'

        ELSE 'Small Order'
    END AS OrderCategory

FROM Orders;
```

Conditional Calculation

CASE can return numbers too.

Example:

```sql
SELECT
    EmployeeName,
    Salary,

    CASE
        WHEN Department = 'Sales'
            THEN Salary * 0.10

        ELSE Salary * 0.05
    END AS Bonus

FROM Employees;
```

So CASE is not only for text labels.

It can return:

```
EXAMPLE / FLOW

Numbers
Dates
Strings
Other compatible expressions
```

Conditional Aggregation

A very useful advanced pattern:

```sql
SELECT
    SUM(
        CASE
            WHEN Status = 'Paid'
            THEN Amount
            ELSE 0
        END
    ) AS PaidAmount
FROM Orders;
```

This calculates only paid amounts.

Another:

```sql
SELECT
    COUNT(
        CASE
            WHEN Status = 'Pending'
            THEN 1
        END
    ) AS PendingOrders
FROM Orders;
```

CASE Ordering Rule

SQL Server evaluates matching conditions in order.

Example:

Wrong logic:

```sql
CASE
    WHEN Salary >= 50000 THEN 'Medium'
    WHEN Salary >= 100000 THEN 'High'
END
```

A salary of:

EXAMPLE / FLOW

```
120000
```

already satisfies:

EXAMPLE / FLOW

```
Salary >= 50000
```

so it may return:

EXAMPLE / FLOW

```
Medium
```

The better order:

```sql
CASE
    WHEN Salary >= 100000 THEN 'High'
    WHEN Salary >= 50000 THEN 'Medium'
    ELSE 'Low'
END
```

Put more specific/higher-priority conditions first.

CASE Is an Expression

Important:

`CASE` in T-SQL is an expression, not a procedural flow-control statement.

It returns a value.

For procedural branching, SQL Server also has `IF...ELSE`, which is a separate topic.

CASE Cheat Sheet

EXAMPLE / FLOW

```
Simple CASE
→ One value compared with alternatives
```

EXAMPLE / FLOW

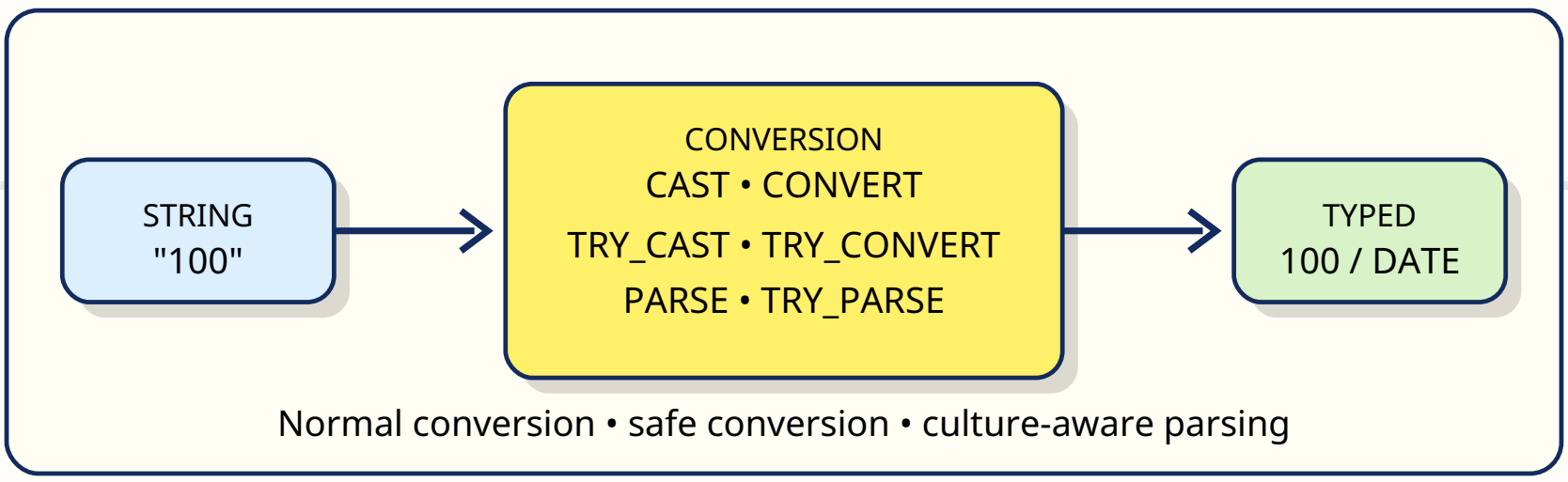
```
Searched CASE
→ Multiple Boolean conditions
```

Uses:

EXAMPLE / FLOW

```
Conditional labels
Conditional calculations
Conditional aggregation
Custom sorting
Display transformation
```

CONVERSION FUNCTIONS



SQL Server frequently needs values converted between data types.

Important functions:

EXAMPLE / FLOW

```
CAST
CONVERT
TRY_CAST
TRY_CONVERT
PARSE
TRY_PARSE
```

CAST

Converts an expression to another data type.

Syntax

SQL

```
CAST(
    expression
    AS datatype
)
```

Example:

SQL

```
SELECT CAST(
    '100'
    AS INT
);
```

Result:

EXAMPLE / FLOW

```
100
```

Another:

SQL

```
SELECT CAST(
    123.45
    AS INT
);
```

Result:

EXAMPLE / FLOW

```
123
```

CONVERT

Also converts values.

Syntax

SQL

```
CONVERT(
    datatype,
    expression,
    style
)
```

Example:

SQL

```
SELECT CONVERT(
    INT,
    '100'
);
```

Result:

EXAMPLE / FLOW

```
100
```

`CONVERT` is particularly useful with SQL Server date formatting styles.

Example:

SQL

```
SELECT CONVERT(
    VARCHAR(10),
    GETDATE(),
    23
);
```

Possible result:

EXAMPLE / FLOW

```
2026-09-14
```

CAST vs CONVERT

EXAMPLE / FLOW

```
CAST
→ Standard SQL-style conversion

CONVERT
→ SQL Server conversion
→ Supports style argument
```

For simple conversions:

SQL

```
CAST('100' AS INT)
```

and:

SQL

```
CONVERT(INT, '100')
```

serve the same basic purpose.

TRY_CAST

Attempts a conversion.

If the conversion fails:

EXAMPLE / FLOW

```
Returns NULL
```

instead of normal conversion failure.

Example:

SQL

```
SELECT TRY_CAST(
    '100'
    AS INT
);
```

Result:

EXAMPLE / FLOW

```
100
```

Invalid:

SQL

```
SELECT TRY_CAST(
    'ABC'
    AS INT
);
```

Result:

EXAMPLE / FLOW

```
NULL
```

TRY_CONVERT

Same basic idea:

SQL

```
SELECT TRY_CONVERT(
    INT,
    'ABC'
);
```

Result:

EXAMPLE / FLOW

```
NULL
```

Useful when processing messy imported data.

PARSE

PARSE converts strings into date/time or numeric values using .NET/culture-aware parsing.

Example:

SQL

```
SELECT PARSE(
    '12/31/2026'
    AS DATE
    USING 'en-US'
);
```

It can be useful for culture-specific input.

Another culture may interpret dates differently.

When to Use PARSE

Use mainly when:

EXAMPLE / FLOW

```
Input is a string
+
Culture-specific interpretation matters
```

Examples:

EXAMPLE / FLOW

```
US-style date text
European formatted numbers
Culture-specific currency/numeric strings
```

When Not to Use PARSE

Don't use PARSE for normal high-volume type conversion when CAST or CONVERT is sufficient.

PARSE relies on .NET parsing and is generally more expensive.

Prefer:

EXAMPLE / FLOW

```
CAST
CONVERT
```

for normal database conversions.

TRY_PARSE

Like PARSE, but returns NULL when parsing fails.

Example:

SQL

```
SELECT TRY_PARSE(
    '12/31/2026'
    AS DATE
    USING 'en-US'
);
```

Invalid:

SQL

```
SELECT TRY_PARSE(
    'Not a date'
    AS DATE
    USING 'en-US'
);
```

returns:

EXAMPLE / FLOW

```
NULL
```

CAST / CONVERT / PARSE Comparison

FunctionFailureMain Use		
CAST	Error	Standard conversion
CONVERT	Error	SQL Server conversion/styles
TRY_CAST	NULL	Safe conversion
TRY_CONVERT	NULL	Safe SQL Server conversion
PARSE	Error	Culture-aware string parsing
TRY_PARSE	NULL	Safe culture-aware parsing

Memory:

EXAMPLE / FLOW

```
CAST / CONVERT
→ Normal conversion

TRY_
→ Return NULL if conversion fails

PARSE
→ Culture-aware string interpretation
```

Important Conversion Example

Suppose imported data contains:

EXAMPLE / FLOW

```
Amount
------
100
250
ABC
500
```

This:

SQL

```
SELECT CAST(
    Amount
    AS INT
)
FROM ImportedData;
```

can fail on:

EXAMPLE / FLOW

```
ABC
```

Instead:

SQL

```
SELECT TRY_CAST(
    Amount
    AS INT
)
FROM ImportedData;
```

returns conceptually:

EXAMPLE / FLOW

```
100
250
NULL
500
```

You can then find invalid rows:

SQL

```
SELECT *
FROM ImportedData
WHERE TRY_CAST(Amount AS INT) IS NULL;
```

FUNCTIONS INSIDE EXPRESSIONS

Functions can be combined.

Example:

SQL

```
SELECT
    UPPER(
        CONCAT(
            FirstName,
            ' ',
            LastName
        )
    ) AS FullName
FROM Employees;
```

Flow:

EXAMPLE / FLOW

```
FirstName + LastName
        ↓
CONCAT
        ↓
UPPER
        ↓
Final result
```

Functions with CASE

Example:

SQL

```
SELECT
    EmployeeName,

    CASE
        WHEN DATEDIFF(
                YEAR,
                HireDate,
                GETDATE()
             ) >= 10
        THEN 'Long Service'

        ELSE 'Regular'
    END AS ServiceCategory

FROM Employees;
```

Functions and expressions can therefore be combined to build sophisticated query logic.

Complete Practical Example

Suppose we have:

EXAMPLE / FLOW

```
Employees
```

with:

EXAMPLE / FLOW

```
EmployeeId
FirstName
LastName
Salary
Bonus
HireDate
Email
Department
```

Query:

SQL

```
SELECT
    EmployeeId,

    CONCAT_WS(
        ' ',
        FirstName,
        LastName
    ) AS FullName,

    UPPER(Department) AS Department,

    ROUND(Salary, 2) AS Salary,

    ISNULL(Bonus, 0) AS Bonus,

    Salary + ISNULL(Bonus, 0)
        AS TotalCompensation,

    YEAR(HireDate)
        AS HireYear,

    DATEDIFF(
        YEAR,
        HireDate,
        GETDATE()
    ) AS ApproxYearsOfService,

    CASE
        WHEN Salary >= 100000
            THEN 'High Salary'

        WHEN Salary >= 60000
            THEN 'Medium Salary'

        ELSE 'Standard Salary'
    END AS SalaryCategory,

    LOWER(Email)
        AS NormalizedEmail

FROM Employees;
```

This one query uses:

EXAMPLE / FLOW

```
CONCAT_WS
UPPER
ROUND
ISNULL
YEAR
DATEDIFF
CASE
LOWER
```

Important Rules and Common Mistakes

Do Not Use Functions Unnecessarily on Indexed Filter Columns

This:

SQL

```
WHERE YEAR(OrderDate) = 2026;
```

may make efficient index usage harder.

Prefer:

SQL

```
WHERE OrderDate >= '2026-01-01'
  AND OrderDate <  '2027-01-01';
```

Do Not Confuse LEN and DATALENGTH

EXAMPLE / FLOW

```
LEN
→ Characters

DATALENGTH
→ Bytes
```

Remember CONCAT Handles NULL Differently

SQL

```
CONCAT('A', NULL, 'B')
```

returns:

EXAMPLE / FLOW

```
AB
```

where ordinary string concatenation using + can result in NULL.

Don't Use FORMAT for Heavy Data Processing

`FORMAT` is convenient for display but relatively expensive.

Prefer application formatting or simpler conversion functions where possible.

Do Not Use FLOAT for Exact Financial Calculations

Use:

EXAMPLE / FLOW

```
DECIMAL
```

for exact business values.

Functions such as:

EXAMPLE / FLOW

```
ROUND
```

cannot correct the fundamental approximate-storage behavior of floating-point data.

Be Careful with DATEDIFF for Age

This:

SQL

```
DATEDIFF(
    YEAR,
    BirthDate,
    GETDATE()
)
```

doesn't automatically mean exact age.

It counts year boundaries.

Do Not Automatically Replace NULL with Zero

Before:

SQL

```
ISNULL(Value, 0)
```

ask:

> Does NULL actually mean zero in this business context?

Sometimes:

EXAMPLE / FLOW

```
NULL
```

means:

EXAMPLE / FLOW

```
Unknown
```

which is very different from:

EXAMPLE / FLOW

```
0
```

CASE Conditions Should Be Ordered Carefully

Put the most restrictive/highest conditions first.

Good:

```sql
CASE
    WHEN Salary >= 100000 THEN 'High'
    WHEN Salary >= 50000 THEN 'Medium'
    ELSE 'Low'
END
```

Master Cheat Sheet

String

EXAMPLE / FLOW

```
CONCAT
→ Join text

CONCAT_WS
→ Join with separator

LEFT / RIGHT
→ Extract from side

LEN
→ Character count

DATALENGTH
→ Byte count

TRIM
→ Remove outer spaces

UPPER / LOWER
→ Change case

REPLACE
→ Replace text

SUBSTRING
→ Extract section

STRING_SPLIT
→ String to rows

STRING_AGG
→ Rows to string

CHARINDEX
→ Find text

PATINDEX
→ Find pattern

STUFF
→ Replace section

REVERSE
→ Reverse string

REPLICATE
→ Repeat text
```

Date/Time

EXAMPLE / FLOW

```
GETDATE
→ Current local datetime

SYSUTCDATETIME
→ Current UTC datetime2

DATEADD
→ Add/subtract time

DATEDIFF
→ Calculate difference

DATEPART
→ Numeric component

DATENAME
→ Text component

YEAR/MONTH/DAY
→ Individual components

EOMONTH
→ Last day of month

DATEFROMPARTS
→ Build date
```

Numeric

EXAMPLE / FLOW

```
ROUND
→ Round

ABS
→ Absolute value

CEILING
→ Up

FLOOR
→ Down

POWER
→ Exponent

SQRT
→ Square root

SQUARE
→ Square

SIGN
→ -1 / 0 / 1

RAND
→ Random-like value

PI
→ π
```

NULL

EXAMPLE / FLOW

```
IS NULL
→ Check NULL

ISNULL
→ Replace NULL

COALESCE
→ First non-NULL

NULLIF
→ NULL when values match
```

Conditional

EXAMPLE / FLOW

```
CASE
→ Conditional result
```

Conversion

EXAMPLE / FLOW

```
CAST
→ Standard conversion

CONVERT
→ SQL Server conversion

TRY_CAST
→ Safe CAST

TRY_CONVERT
→ Safe CONVERT

PARSE
→ Culture-aware parsing

TRY_PARSE
→ Safe culture-aware parsing
```

20 Interview Questions

1. What is a SQL Server function?

A function accepts input, performs an operation, and returns a result. Examples include `UPPER()`, `GETDATE()`, `ROUND()`, and `ISNULL()`.

2. What is an expression?

An expression is a combination of values, columns, operators, and functions that produces a result, such as `Salary * 12`.

3. What is the difference between CONCAT and CONCAT_WS?

`CONCAT` joins values directly. `CONCAT_WS` additionally places a specified separator between values and conveniently skips NULL arguments.

4. What is the difference between LEN and DATALENGTH?

`LEN` returns character count and normally ignores trailing spaces. `DATALENGTH` returns the actual number of bytes used by the value.

5. What is the difference between CHARINDEX and PATINDEX?

`CHARINDEX` finds literal text. `PATINDEX` finds a pattern and supports wildcards such as `%` and character ranges.

6. What is STRING_SPLIT?

`STRING_SPLIT` converts a delimited string into rows. For example, `'IT,HR'` can become two rows: `IT` and `HR`.

7. What is STRING_AGG?

`STRING_AGG` combines values from multiple rows into one delimited string, such as `IT, HR, Finance`.

8. What is the difference between GETDATE and SYSUTCDATETIME?

`GETDATE()` returns current server/local time as `DATETIME`; `SYSUTCDATETIME()` returns precise UTC time as `DATETIME2`.

9. What does DATEADD do?

`DATEADD` adds or subtracts a specified number of date units, such as days, months, or years, from a date.

10. What does DATEDIFF do?

`DATEDIFF` returns the number of specified date-part boundaries between two date/time values. It should not automatically be treated as an exact age calculation.

11. What is the difference between DATEPART and DATENAME?

`DATEPART` returns a numeric value, such as `9` for September. `DATENAME` returns text such as `September`.

12. What is EOMONTH?

`EOMONTH` returns the final calendar date of the month containing the supplied date, optionally with a month offset.

13. What is the difference between ROUND, CEILING, and FLOOR?

`ROUND` performs normal rounding, `CEILING` moves to the next greater integer, and `FLOOR` moves to the lower integer.

14. What is ISNULL?

`ISNULL(expression, replacement)` returns the replacement when the first expression is NULL. It is useful for simple SQL Server-specific NULL replacement.

15. What is COALESCE?

`COALESCE` returns the first non-NULL expression from a list, making it useful when several possible fallback values exist.

16. What is NULLIF?

`NULLIF(a,b)` returns NULL when `a` and `b` are equal; otherwise it returns `a`. A common use is preventing divide-by-zero errors.

17. How do aggregate functions treat NULL?

Functions such as `SUM`, `AVG`, `MIN`, `MAX`, and `COUNT(column)` generally ignore NULL values. `COUNT(*)` counts rows regardless of column NULLs.

18. What is the difference between simple CASE and searched CASE?

Simple `CASE` compares one expression against values. Searched `CASE` evaluates separate Boolean conditions such as salary ranges.

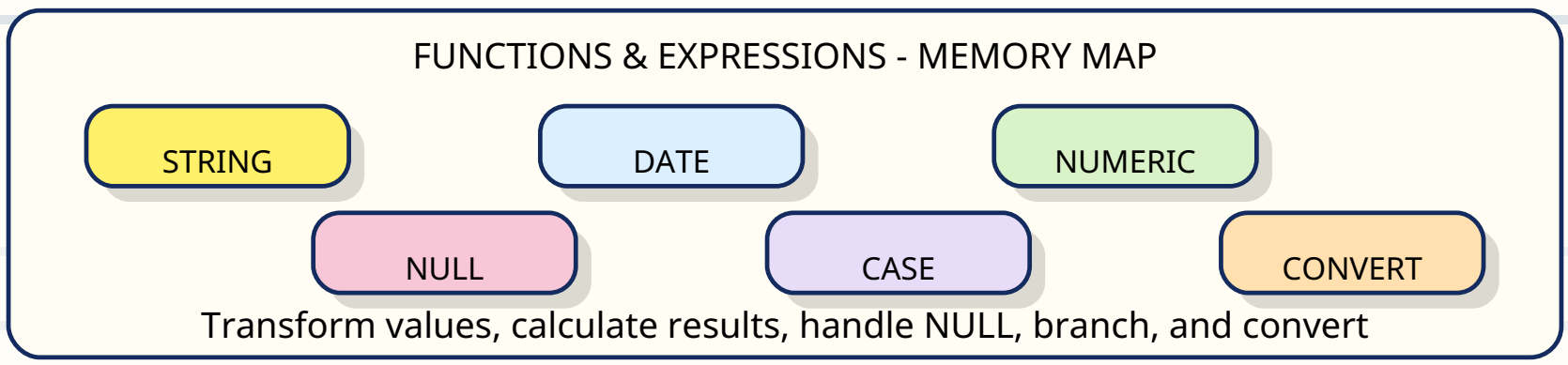
19. What is the difference between CAST and TRY_CAST?

`CAST` normally raises an error when conversion fails. `TRY_CAST` returns NULL when the conversion cannot be performed.

20. What is the difference between CAST, CONVERT, and PARSE?

`CAST` provides standard type conversion, `CONVERT` adds SQL Server-specific formatting styles, and `PARSE` is mainly for culture-aware conversion of strings to date/time or numeric values.

Final Revision



Remember the purpose of each group:

EXAMPLE / FLOW

```
String Functions
→ Manipulate text

Date Functions
→ Work with dates and times

Numeric Functions
→ Perform mathematical operations

NULL Functions
→ Handle missing values

CASE
→ Apply conditional logic

Conversion Functions
→ Change one data type into another
```

The easiest overall rule is:

Use SQL Server functions to transform or calculate values, but keep filtering expressions simple when possible, especially on indexed columns, and only use formatting or conversion where it is genuinely required.